BTS CIEL ~ Initiation langage C avec Arduino

Nom : _____

Initiation à l'utilisation de l' « Arduino Uno » avec un kit de capteurs et d'actionneurs « Grove Starter Kit »

Objectifs :

- Prendre en main une plateforme de prototypage électronique facile à utiliser ainsi qu'un Kit de raccordement rapide de capteurs et d'actionneurs Grove.
- Apprendre le développement logiciel dans un contexte de système embarqué.

Prérequis :

- Connaître les bases de numération et codage
- Savoir utiliser un multimètre

Outils mis à disposition :

- 1 ordinateur équipé de l'IDE arduino
- 1 carte « Arduino Uno » + câble USB type A/B
- 1 kit « Grove Starter »
- 1 multimètre avec fonction « test diode » FLUKE 117

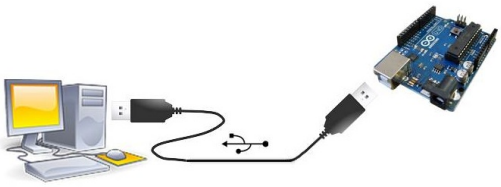
- Vérifier le contenu du Kit à chaque prise en main et à chaque entreposage.
-  Signaler les éléments manquants à votre enseignant.

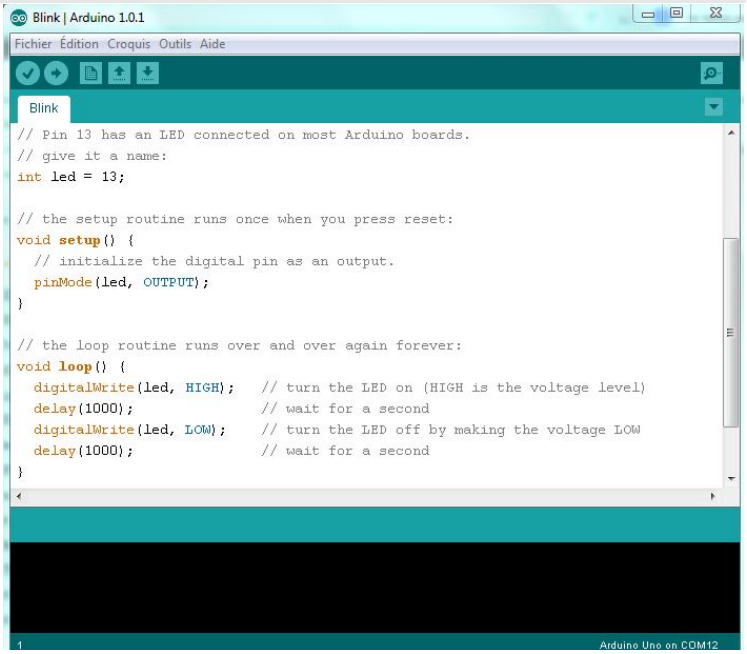
Aspect logiciel de l'Arduino Uno

L'environnement de développement intégré (IDE) « Arduino » qui accompagne les Arduino utilise un langage basé sur les langages C / C++.

Premier pas dans l'IDE Arduino

Voici à quoi ressemble l'IDE, ici avec le programme de démonstration Blink :
Ce premier programme se contentera de faire clignoter la LED jaune témoin qui se trouve près du connecteur D13 de la carte Arduino Uno.
Avant de télécharger le programme dans la carte, il faut relier l'ordinateur et la carte via le câble USB :

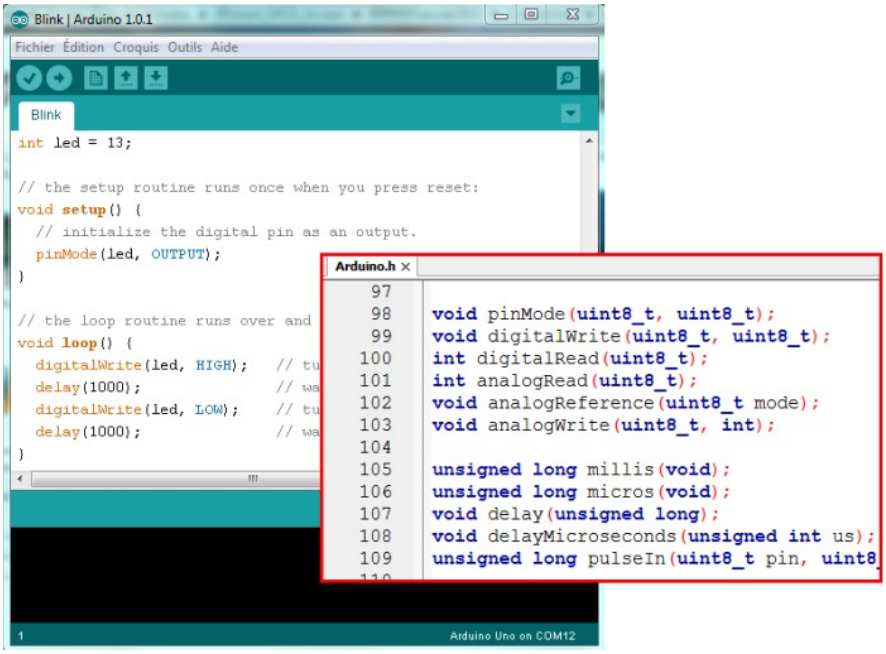




Il faut ensuite sélectionner le bon typer de carte, ainsi que le bon port de communication, via le menu Outils.

Le langage de programmation C/C++

Le « langage » arduino est en réalité du C++, qui a été enrichi de fonctions spécifiques à Arduino : Lorsque vous appelez une de ces fonctions Arduino (non standard C/C++), vous faites appel en toute transparence à une ou plusieurs bibliothèques rédigées en C ou C++ qui seront incluses à la compilation.



En enveloppant le langage C/C++ de cette manière, les concepteurs de l'IDE ont pu simplifier sa syntaxe et l'adapter aux possibilités de la carte. De nombreuses fonctionnalités de haut niveau sont ainsi proposées à l'utilisateur novice qui n'a plus à se soucier de la logique interne du microcontrôleur.

Revenons au programme de démonstration **Blink** :

```
/*
  Commentaire multi-lignes :
  Symboles « ouvrant » indiquant un début de commentaire long
  Blink (Clignotement = Nom donné au programme)
  Allume la LED 13 pendant une seconde, puis l'éteint pendant une seconde, de façon répétée.
  Cet exemple de code est dans le domaine public.
 Symboles « fermants » indiquant une fin de commentaire long
*/

// Commentaire monoligne :  Les symboles « // » indiquent un commentaire jusqu'à la ligne suivante

// La LED 13 est reliée à la broche 13 de la plupart des cartes Arduino
// Donnons-lui un nom (pinLed) :
int pinLed = 13;

// La routine de configuration « setup » s'exécute une seule fois après l'appui
// sur le bouton RESET ou au démarrage de l'Arduino :
void setup()
{
  // Initialisation de la broche 13 ( nommée led ) comme une sortie
  pinMode(pinLed, OUTPUT);
}

// La routine « loop » routine s'exécute en boucle jusqu'à l'arrêt de l'Arduino:
void loop()
{
  digitalWrite(pinLed, HIGH);   // Allume la LED (HIGH correspond à la tension Haute)
  delay(1000);                  // Attendre 1000 ms soit 1 seconde
  digitalWrite(pinLed, LOW);    // Eteint la LED (LOW correspond à la tension Basse)
  delay(1000);                  // Attendre 1000 ms soit 1 seconde
}
```

La structure d'un programme Arduino, rédigée dans l'éditeur, ressemble à ceci et doit toujours comporter les fonctions setup() et loop() :

```
// Définition des constantes, variables globales
// directives de compilation define, include, etc.

void setup()
{
  // initialisation des ressources de la carte,
  // configuration des entrées/sorties,
  // définition de la vitesse de fonctionnement du port série, etc.
  // setup() n'est exécuté qu'une seule fois.
}

void loop()
{
  // les instructions contenues ici sont exécutées indéfiniment en boucle
  // Seule une coupure de l'alimentation de la carte ou un appui sur le bouton Reset
  // permet de quitter le programme.
}
```

Voici les principales fonctions utilisées pour la gestion des entrées/sorties de la carte :

Fonction	Description (partielle)
pinMode()	Configuration des broches numériques en entrée (INPUT) ou en sortie (OUTPUT).
digitalWrite()	Pour une broche numérique configurée en sortie, cette instruction permet de mettre son niveau logique à HAUT ou BAS.
digitalRead()	Lecture du niveau logique (HAUT ou BAS) d'une broche numérique configurée en entrée.
analogWrite()	La carte ne possède pas de Convertisseur Numérique Analogique. Il n'est donc théoriquement pas possible d'obtenir une tension continue qui soit l'image d'une valeur numérique (pas de sortie analogique). Il est par contre possible d'en reproduire le comportement en générant un signal rectangulaire (PWM ou MLI) de rapport cyclique réglable. Beaucoup de composants (LED, moteur électrique à courant continu) agissent comme des filtres passe-bas et fonctionnent comme s'ils étaient alimentés sous une tension continue égale à la valeur moyenne du signal.
analogRead()	Lecture de la valeur de la tension présente sur une entrée analogique (A0 à A5 sur la carte Uno). La fonction retourne la valeur issue de la Conversion Analogique Numérique, soit une valeur comprise entre 0 et 1023 (convertisseur 10 bits).

Tout ce que vous avez toujours voulu savoir sans jamais oser le demander se trouve sur le site arduino, section « Language Reference» arduino.cc/reference

Ainsi, dans la boucle infinie loop() :

```
void loop()
{
  digitalWrite(led, HIGH);   // Allume la LED (HIGH correspond à la tension Haute)
  delay(1000);               // Attendre 1000 ms soit 1 seconde
  digitalWrite(led, LOW);    // Eteint la LED (LOW correspond à la tension Basse)
  delay(1000);               // Attendre 1000 ms soit 1 seconde
}
```

La broche n° 13, reliée à la LED intégrée à la carte, passe alternativement du niveau logique HAUT (HIGH) au niveau logique BAS (LOW) après un délai d'une seconde, ce qui la fait clignoter.

➲ Copier l'intégralité du code du programme « Blink » dans l'éditeur de l'IDE Arduino puis téléverser le programme dans l'Arduino



NB. L'éditeur procédera à la compilation préalable du code avant de le téléverser s'il ne contient pas d'erreur dans l'Arduino.

Q1. Que constatez-vous au niveau de la LED 13 ?

R1.

➲ Modifier le programme « Blink » afin d'obtenir le clignotement de la LED 13 indiqué ci-après puis téléverser le programme modifié dans l'Arduino

- Allumée pendant une demi-seconde
- Eteinte pendant trois secondes
- Répéter 3 fois :
  - Allumée pendant une demi-seconde
  - Eteinte pendant une demi-seconde

Q2. Fournir la copie d'écran de la routine « loop » du programme modifié.

R2.

➲ Consulter la page https://simple-duino.com/les-boucles-arduino/

Q3. Comment pouvez-vous améliorer votre programme ?

R3.

➲ Réaliser la simplification du programme puis téléverser le programme modifié dans l'Arduino

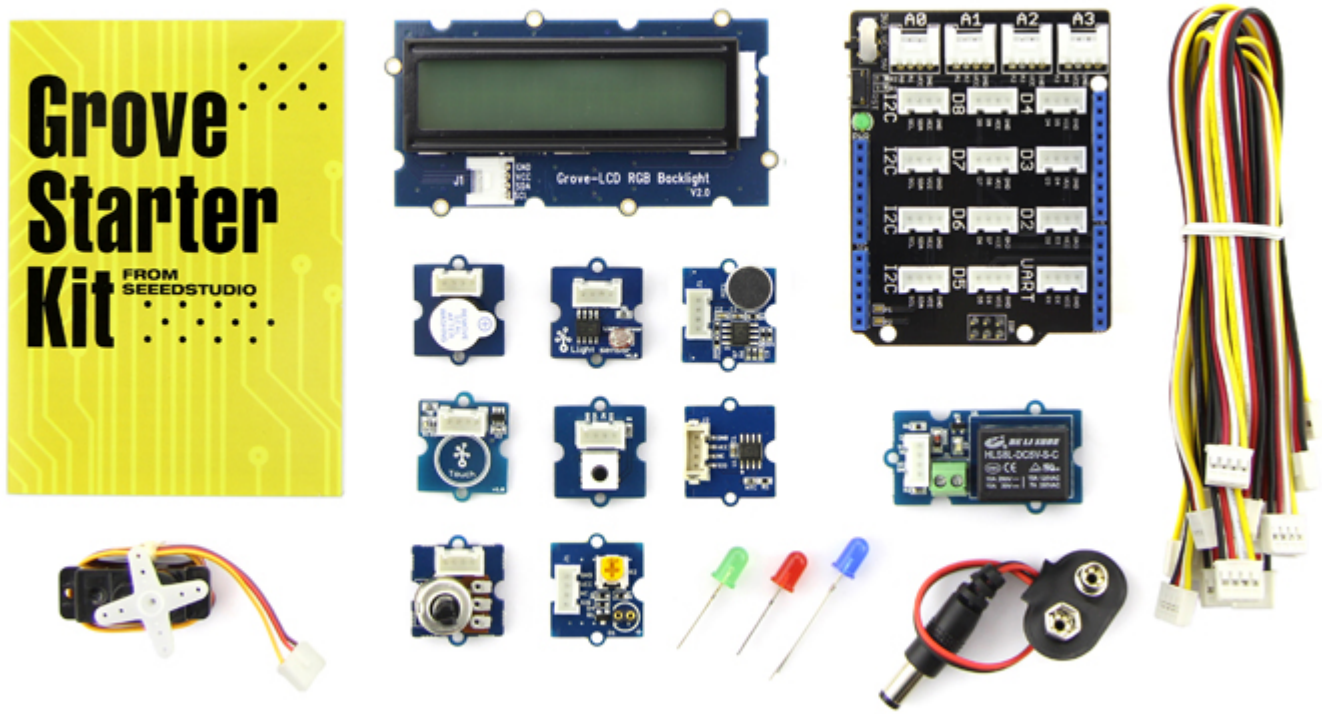
Q4. Proposer une algorigramme du programme simplifié.

R4.

Q5. Fournir la copie d'écran de la routine « loop » du programme modifié.

R5.

Faisons connaissance avec le kit Grove Starter Kit

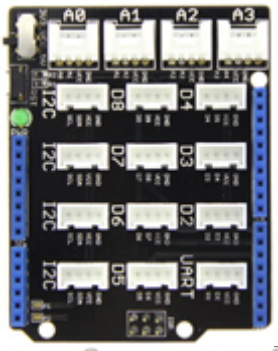


➲ Consulter la ressource http://wiki.seeedstudio.com/Grove_Starter_Kit_v3/

➲ Vérifier que le Starter kit Grove qui vous a été fourni est complet.
⚠ Signaler les éléments manquants à votre enseignant.

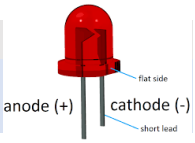
➲ Compléter la colonne « Utilité » du tableau ci-après :

Liste des composants

Part / English	Qty	Photo	Elément / Français	Utilité
Base Shield	1		Grove - Shield Grove	
Grove - LCD RGB Backlight	1		Grove – Ecran LCD RGB rétroéclairé	
Grove - Smart Relay	1		Grove – Relais pouvoir de coupure du relais est de 35 V / 1 A	
Grove - Buzzer	1		Grove – Buzzer	

Grove - Sound Sensor	1		Grove – Capteur sonore (microphone)	
Grove - Touch Sensor	1		Grove – Capteur tactile	
Grove - Rotary Angle Sensor	1		Grove – Capteur d'angle (potentiomètre)	
Grove - Temperature Sensor	1		Grove – Capteur de Température	
Grove - Light Sensor	1		Capteur de lumière	
Grove - Button	1		Grove – Bouton	
Grove - LED	1		Grove – Led (Support de LED)	
DIP LED Blue-Blue	1		Led Bleue	
DIP LED Green-Green	1		Led verte	
DIP LED Red-Red	1		Led rouge	
Grove - Mini Servo	1		Mini servomoteur	
Grove Cables	10		Câbles Grove	
9V to Barrel Jack Adapter	1		Connecteur "DC" pile 9V vers Jack	
Grove starter kit Manual	1		Petit manuel (en Anglais)	
Green Plastic Box	1		Boîte de rangement	

Manipulation du shield Grove et du module LED



➲ Consulter les instructions données sur ce lien
http://wiki.seeedstudio.com/Grove_Starter_Kit_v3/#grove-led



➲ Enficher une des 3 LEDs du Kit (rouge, verte ou bleue) sur le support tulipe de module Grove LED « LED Socket » en respectant la polarité attendue.

➲ Charger le programme « Grove_LED.ino » dans l'IDE Arduino.

```
// Demo for Grove - Starter V2.0
// Author: Loovee  2013-3-10
// Pulses the Grove - LED with a "breathing" effect.
// Connect the Grove - LED to the socket marked D3

// Defines the pin to which the LED is connected.
// Any pin that supports PWM can also be used:
// 3, 5, 6, 9, 10, 11
const int pinLed    = 5;

// Define the delay for the "breathing" effect; change this
// to a smaller value for a faster effect, larger for slower.
const int BREATH_DELAY = 5; // milliseconds

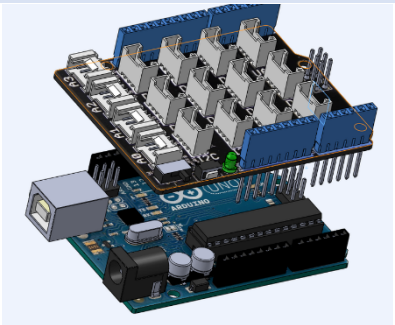
void setup()
{
   // Configure the LED's pin for output signals.
   pinMode(pinLed, OUTPUT);
}

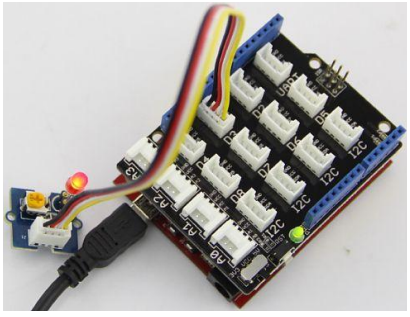
void loop()
{
   for(int i=0; i<=255; i++)
   {
      analogWrite(pinLed, i);
      delay(BREATH_DELAY);
   }
   delay(100);

   for(int i=255; i>=0; i--)
   {
      analogWrite(pinLed, i);
      delay(BREATH_DELAY);
   }
   delay(500);
}
```

➲ ⚠ **Débrancher les sources d'alimentation de la carte Arduino UNO (prise USB, alimentation externe).**

➲ Enficher le shield Grove sur la carte Arduino en faisant attention à ne pas tordre les connecteurs.





➲ Raccorder le module Grove LED sur le port numérique indiqué dans le programme « Grove_LED.ino ».

Q6. Sur quel port avez-vous raccordé le module Grove LED ?

R6.

➲ Rebrancher la prise USB entre le PC et la carte Arduino UNO.

➲ Téléverser le code du programme « Grove_LED.ino » dans l'Arduino UNO.

Q7. Que constatez-vous ?

R7.

Q8. Expliquer en détail le fonctionnement de la boucle suivante :

```
for(int i=0; i<256; i++)
{
    analogWrite(pinLed, i);
    delay(BREATH_DELAY);
}
```

R8.

Q9. Expliquer en détail le fonctionnement de la boucle suivante :

```
for(int i=254; i>=0; i--)
{
    analogWrite(pinLed, i);
    delay(BREATH_DELAY);
}
```

R9.

➲ Vérifier le contenu du Kit à chaque entreposage.

➲ ⚠ Signaler les éléments manquants à votre enseignant.

Pour en savoir plus

- Ressource en ligne - Understanding Arduino UNO Hardware Design